Academic Writing Coach: Week 5 Check-in: Flow and Architecture

Track: Writing Assistant | Core HAI Principle: Transparency | Document status: Week 5 deliverable | Date: November 2026

1. What this report shows

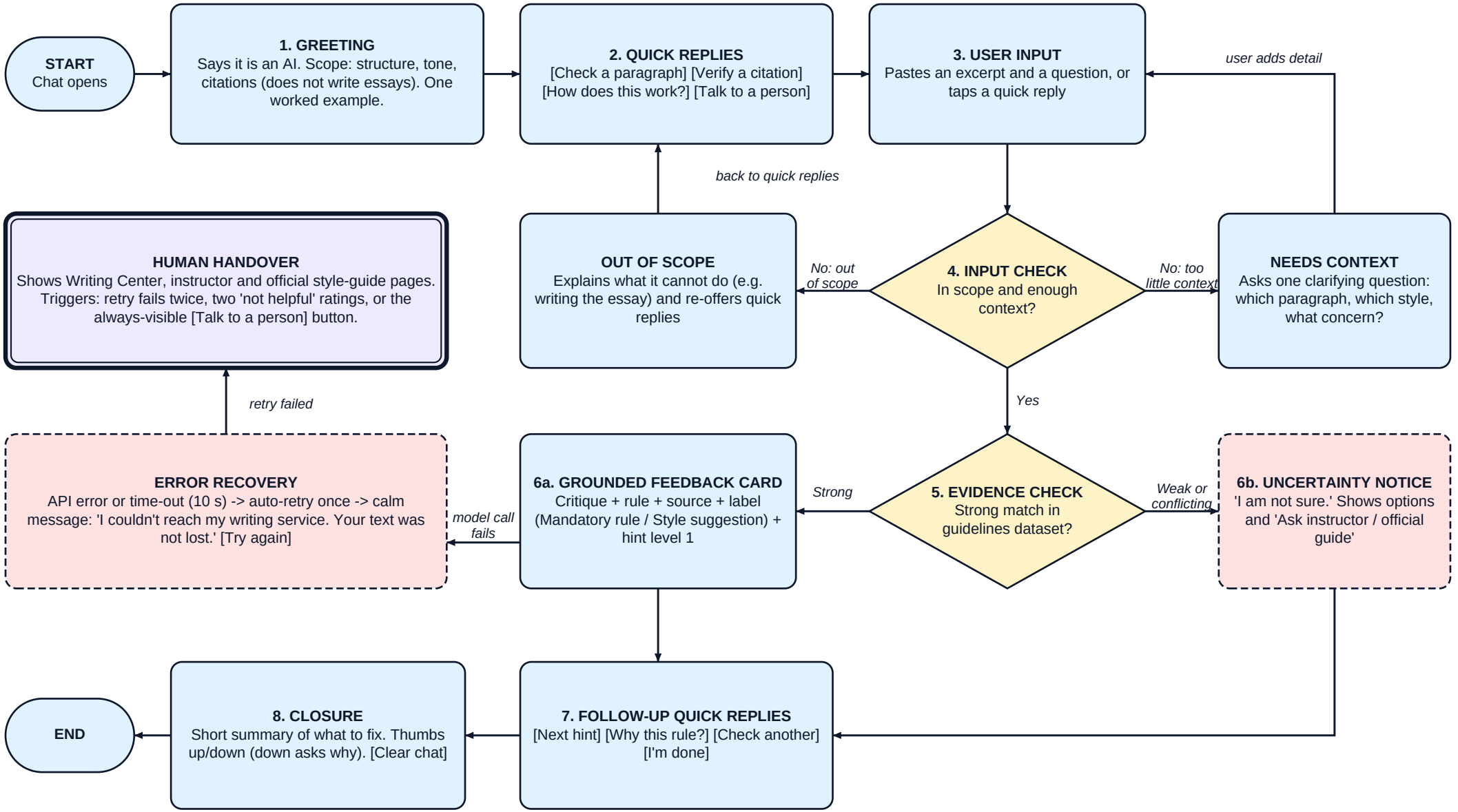
This report contains the five outputs for this check-in: (1) the conversation flow diagram, (2) the system architecture diagram with the Transparency feature marked, (3) the Shneiderman's 8 Golden Rules checklist, (4) the first model response from our own backend, and (5) the prompt log entries for the week.

Design decisions carried over from earlier weeks: a hybrid of Conversational Mentoring and Rule-Based Verification, a FastAPI backend, and grounding on **academic_guidelines_dataset.json** (synthetic, 40 target cases, no real student data).

Reading guide for diagrams

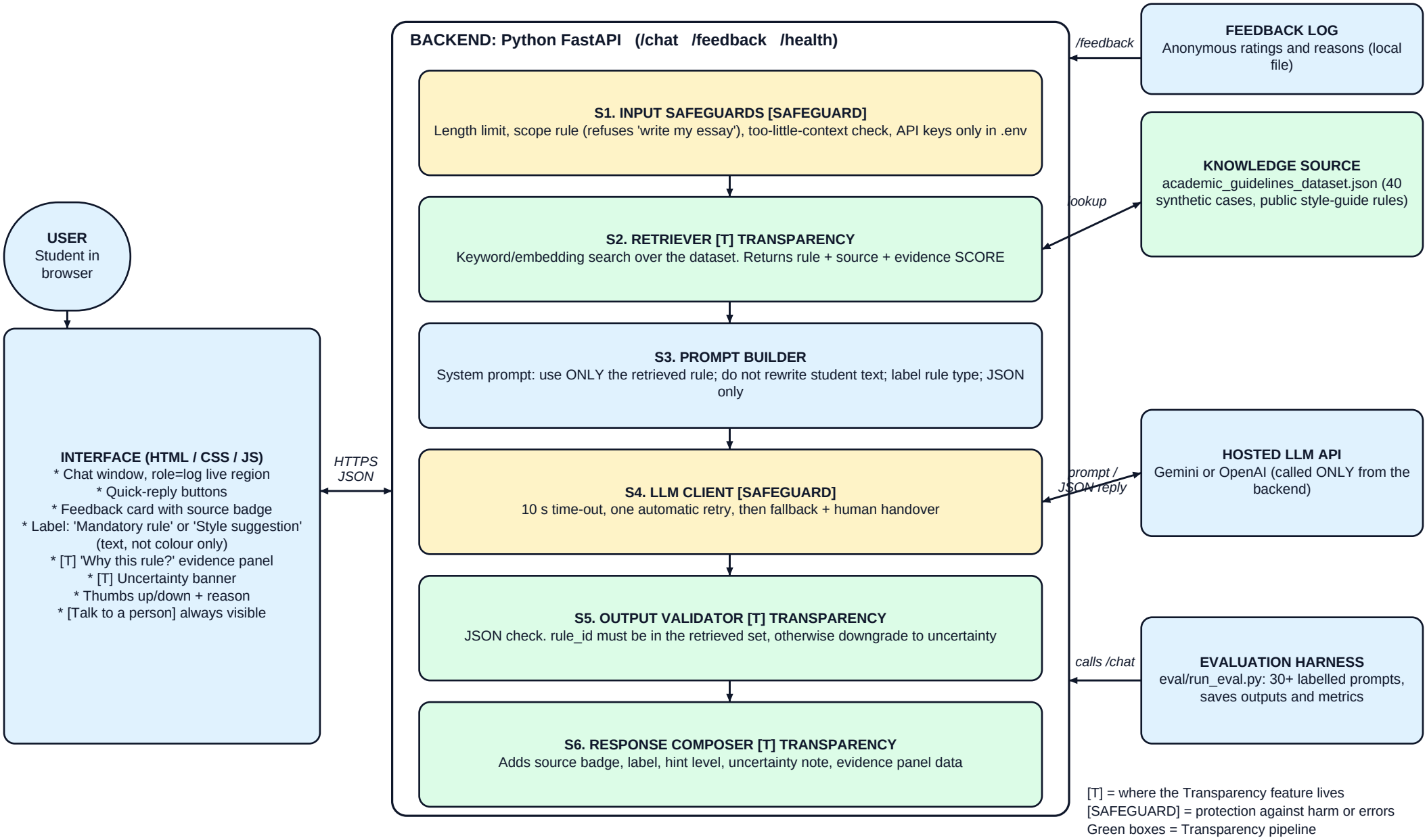
- Rounded rectangle = a step the system or user performs. Diamond = a decision. Oval = start or end.
- Dashed border = error or uncertainty path. Double thick border = human handover.
- Meaning is always written as text inside the shape. Colour is only decoration, so the diagrams still work in black and white.

2. Conversation flow diagram



'Check another' returns to step 2. 'Talk to a person' is reachable from every screen. After handover the chat shows a short summary and closes (step 8).

3. System architecture diagram



3.1 Where the Transparency feature lives

The principle is implemented inside the pipeline, not only on the screen. Three components cooperate:

- **S2 Retriever** returns the rule, its source and an evidence score. The score is used to decide whether the system is sure enough to answer.
- **S5 Output validator** rejects any model answer whose rule_id is not in the retrieved set. An answer that cannot point to real evidence is replaced by an uncertainty notice.
- **S6 Response composer** sends the real evidence to the interface, so the 'Why this rule?' panel shows what the system actually used. It is never written after the fact.

3.2 Safeguards

- Input: length limit, scope rule, too-little-context check.
- Model: 10 second time-out, one automatic retry, calm fallback message, human handover.
- Output: JSON validation and evidence match. The model is told not to rewrite student text.
- Data and keys: only synthetic or public data. API keys live in a local .env file that is git-ignored; the browser never sees them.

3.3 Failure cases handled in this version

Situation	What the system does	Status
Student asks it to write the essay	Polite refusal, explains scope, re-offers quick replies	out_of_scope
Message too short or unclear	Asks one clarifying question, no critique	needs_context
No strong rule found (low score)	'I am not sure' notice + handover options	uncertain
Model answer cites an unknown rule	Answer hidden, uncertainty notice shown	uncertain
API error, wrong key or time-out	Retry once, then fallback message + handover	error

4. Shneiderman's 8 Golden Rules checklist

Each rule is linked to a screen or a flow decision (step numbers refer to the flow diagram).

Rule	Where we meet it (screen or flow decision)
1. Strive for consistency	Every feedback card has the same layout: critique, rule, source badge, label, hint. Labels are always the same two phrases: 'Mandatory rule' and 'Style suggestion'. Quick replies use the same wording on every screen (steps 2 and 7).
2. Enable shortcuts	Quick-reply buttons for frequent tasks (Check a paragraph, Verify a citation). Advanced users (persona Ziad) can skip the intro and tap 'Show source only'. Keyboard: Enter sends, Tab reaches every button.
3. Offer informative feedback	A 'Checking the style guide...' status appears while waiting. Each answer shows its source and a label. If the system is unsure, the uncertainty notice says so (step 6b). Errors say what happened and what to do (error recovery).
4. Design dialogs that give closure	Step 8 gives a short summary of what to fix, then asks for a thumbs up/down. A handover ends with a clear statement of who to contact next.
5. Prevent errors and handle them simply	Input check (step 4) catches out-of-scope or too-short messages before the model is called. Hallucinated rules are blocked by the output validator. Errors give one plain sentence and a [Try again] button.
6. Permit easy reversal of actions	[Clear chat] button, edit and resend the last message, and 'Show previous hint' as well as 'Next hint'. The system never changes the student's text, so there is nothing to undo in the draft.
7. Support the user's sense of control	The student chooses the style manual (APA, MLA, IEEE), the hint level (step 7) and when to stop. The AI explains but never rewrites. [Talk to a person] is visible on every screen.
8. Reduce short-term memory load	The rule and source stay attached to each feedback card, so the student does not need to remember them. Quick replies show the next possible actions. Plain-language wording, one hint at a time, and a persistent Help button.

5. First model response from our own backend

The call is made by **our FastAPI backend** to a hosted model API. The key is read from an environment variable on the server. A reply typed into a public AI website does not count, so the evidence below must come from running this code.

5.1 Core of the backend call (backend/app/llm.py)

```
def _call_gemini(system, user):
    key = os.getenv("GEMINI_API_KEY")           # never hard-coded
    model = os.getenv("GEMINI_MODEL", "gemini-2.0-flash")
    url = f"https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent"
    body = {"system_instruction": {"parts": [{"text": system}]},
            "contents": [{"role": "user", "parts": [{"text": user}]}],
            "generationConfig": {"temperature": 0.2}}
    r = httpx.post(url, json=body, headers={"x-goog-api-key": key}, timeout=10)
    r.raise_for_status()
    return r.json()["candidates"][0]["content"]["parts"][0]["text"]

def generate(system, user):                     # one call + ONE automatic retry
    for attempt in (1, 2):
        try: return PROVIDERS[provider](system, user)
        except (httpx.HTTPError, LLMError, KeyError, ValueError) as e: last = e
    raise LLMError(str(last))                   # caller shows fallback + handover
```

5.2 How to run it and capture the evidence

```
cd backend
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env          # then put YOUR key in .env (never commit it)
uvicorn app.main:app --reload                         # terminal 1
python scripts/first_call.py                          # terminal 2 -> saves evidence/*.json
```

Screen-record both terminals, or screenshot them, and paste the screenshot in the box below. The script prints the HTTP status, the response time and the full JSON, including the evidence block that feeds the 'Why this rule?' panel.

5.3 What the response looks like (structure)

```
{ "status": "ok",
  "message": "<model feedback grounded in the retrieved rule>",
  "hint": "<one short hint, not a rewrite>",
  "label": "mandatory_rule",
  "evidence": { "rule_id": "CF-001", "rule": "APA 7 author-date citations include ...",
                "source": "APA Publication Manual, 7th ed., Ch. 8 ...", "score": 0.59 },
  "uncertainty_note": null, "latency_ms": <measured> }
```

5.3b Captured output from this backend

```
Local test run of our backend (POST /chat), offline MOCK model, 5 Nov 2026
{ "status": "ok",
  "message": "[MOCK] Check the rule shown in the evidence panel.",
  "hint": "[MOCK] Compare your sentence with the rule.",
  "label": "mandatory_rule",
  "evidence": { "rule_id": "CF-001",
                "rule": "APA 7 author-date citations include the author surname and the year ...",
                "source": "APA Publication Manual, 7th ed., Ch. 8 ...", "score": 0.59 },
  "uncertainty_note": null, "latency_ms": 3 }
```

```
Error path, no API key set (real provider selected):
status = error | "I couldn't reach my writing service. Your text was not lost.
                  You can try again, or contact a person."
```

The [MOCK] text proves the pipeline works end to end (scope check, retrieval, evidence, validation, response). It is not a live model reply: the live call only needs a key in .env and the same code, with the same JSON structure.

5.4 What was tested while building this version

The pipeline was tested offline with a mock model (labelled [MOCK] in its output, so it can never be mistaken for a real reply). All paths returned the expected status: ok, out_of_scope, needs_context, uncertain, and error (a missing API key triggered the fallback correctly). The mock is for tests only. **The first real model response has to be produced by running the steps above with your own key.**

6. Prompt log: Week 5 (Appendix A)

Exact prompt and what was changed after checking the output. Add a new row for any other AI tool the team actually uses.

#	Tool	Exact prompt	What it gave	What we changed and why
5.1	Claude	Build the Week 5 deliverables for our Academic Writing Coach (Transparency, FastAPI backend, academic_guidelines_dataset.jso n): conversation flow with error recovery, fallback and human handover; architecture diagram with the principle feature marked; Shneiderman's 8 Golden Rules checklist; a first backend model call; prompt log. (Translated from the Arabic request.)	Flow and architecture diagrams, an 8-rule checklist, a FastAPI backend (retriever, LLM client, output validator), a 6-case sample dataset, a report and a slide deck.	Fixed overlapping labels in the flow diagram; replaced a star symbol that did not render with [T] markers; enlarged architecture text for slides; added Page X of Y footer; tested every response path offline (ok, out_of_scope, needs_context, uncertain, error); labelled mock output as [MOCK] so it is never mistaken for a live reply.